Godot 4 C# （Windows 专属）外部应用窗口识别 完整落地方案

本方案为**Windows 平台定制**，基于 `user32.dll` 原生 API + C# P/Invoke 实现，包含**基础枚举、精准查找、自身窗口过滤、窗口交互、Godot 可视化、实时更新**全流程，代码可直接复制到工程中运行，附带详细的工程配置、逻辑拆解和避坑指南，兼顾基础使用和实际开发需求。

一、工程前置配置

1.1 环境要求

- Godot 4.x （4.0+ 均可，推荐 4.2/4.3 稳定版）
- 开启 C# 支持（新建工程时选择 **C#** 模板，或在已有工程中启用 Mono）
- .NET 6.0+ （Godot 4 内置 Mono 环境，无需额外安装，仅需确保工程 SDK 匹配）

1.2 节点结构（极简可运行）

新建 2D 工程，创建如下节点层级（直接在场景编辑器拖拽创建），**所有脚本挂在 `WindowDetector` 上**：

Code block

```
Root (Node2D)
└── WindowDetector (Node2D)  # 核心处理节点，挂C#脚本
```

为什么选 Node2D：支持 `_Draw` 绘制函数，可直接在Godot窗口中绘制外部窗口的矩形/标题，实现可视化预览。

二、核心基础：全量枚举可视外部窗口

2.1 核心逻辑

通过 `EnumWindows` 枚举系统所有顶层窗口 → 回调函数中过滤**可视+有标题**窗口 → 调用 `GetWindowRect` / `GetWindowThreadProcessId` 获取窗口位置/PID → 封装为自定义 `WindowInfo` 类 → 存储到列表供后续使用。

2.2 完整代码（基础枚举版）

新建 C# 脚本 `WindowDetector.cs`，替换所有内容为以下代码（**带逐行逻辑注释**）：

Code block

```
using Godot;
using System;
using System.Collections.Generic;
using System.Runtime.InteropServices;
using System.Text;
using System.Diagnostics; // 用于获取自身PID
using System.Linq;        // 用于LINQ过滤，精准查找必备

// 继承Node2D，支持绘制和场景树生命周期
public partial class WindowDetector : Node2D
{
    #region 第一步：封装Windows user32.dll 核心API
    // 枚举所有顶层窗口（核心API）
    [DllImport("user32.dll")]
    private static extern bool EnumWindows(EnumWindowsProc lpEnumFunc, IntPtr lParam);
    // 窗口枚举回调委托（签名必须和原生一致，否则崩溃）
    private delegate bool EnumWindowsProc(IntPtr hWnd, IntPtr lParam);

    // 判断窗口是否可视（过滤隐藏窗口）
    [DllImport("user32.dll")]
    private static extern bool IsWindowVisible(IntPtr hWnd);
    // 获取窗口标题（必须用StringBuilder接收，C#字符串不可变）
    [DllImport("user32.dll", CharSet = CharSet.Auto)]
    private static extern int GetWindowText(IntPtr hWnd, StringBuilder lpString, int nMaxCount);
    // 获取窗口屏幕坐标（原生RECT结构体）
    [DllImport("user32.dll")]
    private static extern bool GetWindowRect(IntPtr hWnd, out RECT lpRect);
    // 获取窗口所属进程ID（PID）和线程ID
    [DllImport("user32.dll")]
    private static extern uint GetWindowThreadProcessId(IntPtr hWnd, out uint lpdwProcessId);

    // Windows原生矩形结构体（和GetWindowRect配套，内存布局必须Sequential）
    [StructLayout(LayoutKind.Sequential)]
    private struct RECT
    {
        public int Left;   // 左上角X
        public int Top;    // 左上角Y
        public int Right;  // 右下角X
        public int Bottom; // 右下角Y
    }
    #endregion

```

```
    #region 第二步：自定义窗口信息类（封装原生数据，Godot友好）
    /// <summary>
    /// 单窗口信息封装，存储所有常用属性
    /// </summary>
    public class WindowInfo
    {
        public IntPtr Hwnd { get; set; } // 窗口句柄（唯一标识，操作窗口的核心）
        public string Title { get; set; } // 窗口标题
        public Rect2I Rect { get; set; } // Godot矩形（屏幕坐标，直接用于绘制/计算）
        public uint Pid { get; set; } // 所属进程ID（用于过滤自身/指定进程）
        public int Width => Rect.Size.X; // 窗口宽度（快捷属性）
        public int Height => Rect.Size.Y; // 窗口高度（快捷属性）
    }
    #endregion

    #region 第三步：全局变量（工程中可直接访问）
    /// <summary>
    /// 所有过滤后的可视外部窗口列表（排除自身）
    /// </summary>
    public List<WindowInfo> AllExternalWindows = new List<WindowInfo>();
    /// <summary>
    /// 当前选中的外部窗口（用于可视化/交互）
    /// </summary>
    public WindowInfo SelectedWindow { get; set; }
    /// <summary>
    /// Godot自身进程PID（用于过滤自身窗口）
    /// </summary>
    private uint _selfPid;
    /// <summary>
    /// 窗口标题缓冲区大小（建议256/512，避免标题截断）
    /// </summary>
    private const int TitleBufferSize = 256;
    #endregion

    #region 第四步：初始化（_Ready中执行核心枚举）
    public override void _Ready()
    {
        // 1. 获取自身PID，后续过滤自身窗口
        _selfPid = (uint)Process.GetCurrentProcess().Id;
        // 2. 首次枚举外部窗口
        EnumerateExternalWindows();
        // 3. 打印枚举结果（Godot控制台查看）
```

2.3 运行测试

将脚本挂在 `WindowDetector` 节点上，点击**运行**，在 Godot **控制台**中可看到枚举结果，包含所有外部窗口的**句柄、标题、屏幕位置、PID**，无Godot自身窗口（已过滤）。

2.4 基础版核心避坑点

1. **回调委托签名**：`EnumWindowsProc` 的参数和返回值必须和原生一致（`IntPtr hWnd, IntPtr lParam` → `bool`），否则会导致Godot崩溃；
2. **标题接收方式**：必须用 `StringBuilder`，不能直接用 `string`（C# 字符串不可变，Win32 API 无法写入）；
3. **坐标转换**：Win32 `RECT` 是**右下角坐标**，Godot `Rect2I` 是**左上角+宽高**，必须通过 `Right-Left` / `Bottom-Top` 计算宽高；
4. **自身PID过滤**：通过 `Process.GetCurrentProcess().Id` 获取，转换为 `uint` 匹配（Win32 API 返回的PID是uint类型）。

三、进阶功能1：精准查找窗口（比全量枚举更高效）

实际开发中很少需要全量枚举，更多是**按标题模糊匹配/精准匹配、按PID查找、按句柄查找**，基于基础版的 `AllExternalWindows` 列表，通过 **LINQ 过滤** 实现，无需重新调用系统API，效率更高。

3.1 新增精准查找方法（添加到 `WindowDetector.cs` 中，任意区域均可，建议放在辅助方法后）

Code block

```
#region 进阶功能：精准查找窗口（LINQ过滤）
/// <summary>
/// 按标题模糊查找（忽略大小写，如查找包含“Chrome”/“微信”的窗口）
/// </summary>
/// <param name="titlePart">标题关键字</param>
/// <returns>匹配的窗口列表</returns>
public List<WindowInfo> FindWindowsByTitle(string titlePart)
{
    if (string.IsNullOrEmpty(titlePart)) return new List<WindowInfo>();
    return AllExternalWindows
        .Where(w => w.Title.Contains(titlePart, StringComparison.OrdinalIgnoreCase))
        .ToList();
}

/// <summary>
/// 按标题精准查找（完全匹配，区分大小写）
/// </summary>
/// <param name="exactTitle">完整窗口标题</param>
/// <returns>匹配的窗口（无则返回null）</returns>
```

```
public WindowInfo FindWindowByExactTitle(string exactTitle)
{
    return AllExternalWindows.FirstOrDefault(w => w.Title == exactTitle);
}

/// <summary>
/// 按进程ID（PID）查找窗口（如查找某个软件的所有窗口）
/// </summary>
/// <param name="pid">进程ID</param>
/// <returns>匹配的窗口列表</returns>
public List<WindowInfo> FindWindowsByPid(uint pid)
{
    return AllExternalWindows.Where(w => w.Pid == pid).ToList();
}

/// <summary>
/// 按窗口句柄（HWND）查找窗口（唯一匹配）
/// </summary>
/// <param name="hwnd">窗口句柄</param>
/// <returns>匹配的窗口（无则返回null）</returns>
public WindowInfo FindWindowByHwnd(IntPtr hwnd)
{
    return AllExternalWindows.FirstOrDefault(w => w.Hwnd == hwnd);
}
#endregion
```

3.2 调用示例（在 `_Ready` 中添加，测试查找功能）

Code block

```
public override void _Ready()
{
    _selfPid = (uint)Process.GetCurrentProcess().Id;
    EnumerateExternalWindows();
    PrintWindowList();

    // 测试：按标题模糊查找Chrome浏览器
    var chromeWindows = FindWindowsByTitle("Chrome");
    GD.Print($"找到 {chromeWindows.Count} 个Chrome窗口：");
    chromeWindows.ForEach(w => GD.Print(w.Title));

    // 测试：按标题模糊查找微信
    var weChatWindow = FindWindowsByTitle("微信").FirstOrDefault();
    if (weChatWindow != null)
    {
```

```
            GD.Print($"找到微信窗口，句柄：{weChatWindow.Hwnd}，位置：{weChatWindow.Rect}");
            SelectedWindow = weChatWindow; // 选中微信窗口，用于后续可视化
        }
    }
```

四、进阶功能2：外部窗口交互（激活/前置/判断是否存在）

拿到窗口句柄（`HWND`）后，通过 `user32.dll` 额外API实现**激活外部窗口、将窗口前置、判断窗口是否有效**，实现Godot和外部软件的联动（如点击Godot按钮，打开微信/Chrome窗口）。

4.1 封装交互API（添加到 `第一步：封装Windows user32.dll 核心API` 区域中）

Code block

```
// 激活/前置外部窗口（点击Godot按钮时调用）
[DllImport("user32.dll")]
private static extern bool SetForegroundWindow(IntPtr hWnd);
// 判断窗口句柄是否有效（避免操作已关闭的窗口）
[DllImport("user32.dll")]
private static extern bool IsWindow(IntPtr hWnd);
```

4.2 新增交互方法（添加到精准查找方法后）

Code block

```
#region 进阶功能：外部窗口交互
/// <summary>
/// 激活/前置外部窗口（将窗口带到屏幕最前方）
/// </summary>
/// <param name="window">要激活的窗口</param>
/// <returns>是否激活成功</returns>
public bool ActivateWindow(WindowInfo window)
{
    if (window == null || !IsWindow(window.Hwnd)) return false;
    return SetForegroundWindow(window.Hwnd);
}

/// <summary>
/// 按句柄激活窗口
/// </summary>
/// <param name="hwnd">窗口句柄</param>
/// <returns>是否激活成功</returns>
public bool ActivateWindow(IntPtr hwnd)
```

```
{
    if (hwnd == IntPtr.Zero || !IsWindow(hwnd)) return false;
    return SetForegroundWindow(hwnd);
}

/// <summary>
/// 判断窗口是否仍有效（未被关闭）
/// </summary>
/// <param name="window">要判断的窗口</param>
/// <returns>是否有效</returns>
public bool IsWindowValid(WindowInfo window)
{
    return window != null && IsWindow(window.Hwnd);
}
#endregion
```

4.3 调用示例（结合Godot按钮实现点击激活）

1. 在场景中添加 **Button** 节点，命名为 `BtnActivateWeChat`，标题改为**激活微信**；
2. 在 `_Ready` 中绑定按钮点击事件，添加代码：

Code block

```
public override void _Ready()
{
    _selfPid = (uint)Process.GetCurrentProcess().Id;
    EnumerateExternalWindows();
    PrintWindowList();

    // 查找微信窗口
    var weChatWindow = FindWindowsByTitle("微信").FirstOrDefault();
    if (weChatWindow != null)
    {
        SelectedWindow = weChatWindow;
        GD.Print($"找到微信窗口：{weChatWindow.Title}");
    }

    // 绑定按钮点击事件
    Button btnActivate = GetNode<Button>("BtnActivateWeChat");
    btnActivate.Pressed += () =>
    {
        if (SelectedWindow != null)
        {
            bool isSuccess = ActivateWindow(SelectedWindow);
            GD.Print(isSuccess ? "窗口激活成功" : "窗口激活失败");
        }
```

```
        else
        {
            GD.Print("未选中任何窗口");
        }
    };
}
```

运行后点击**激活微信**按钮，微信窗口会直接跳到屏幕最前方（需微信已打开）。

五、进阶功能3：Godot 可视化（绘制外部窗口矩形/标题）

通过 Godot `Node2D` 的 `_Draw` 函数，在Godot窗口中**绘制外部窗口的红色半透明矩形**，并在矩形左上角显示窗口标题，直观看到外部窗口的位置和大小，适合窗口捕获、窗口吸附等场景。

5.1 新增绘制方法（添加到代码中，建议放在全局变量后）

Code block

```
#region 进阶功能：Godot可视化 - 绘制外部窗口矩形/标题
public override void _Draw()
{
    // 若有选中的窗口，绘制可视化矩形和标题
    if (SelectedWindow != null && IsWindowValid(SelectedWindow))
    {
        // 1. 绘制红色半透明矩形（匹配外部窗口的屏幕坐标）
        Color rectColor = new Color(1, 0, 0, 0.3f); // 红底，30%透明度
        DrawRect(SelectedWindow.Rect, rectColor);

        // 2. 绘制窗口标题（在矩形左上角，向上偏移20像素，避免重叠）
        Vector2 titlePos = new Vector2(SelectedWindow.Rect.X, SelectedWindow.Rect.Y - 20);
        Color textColor = Color.White; // 白色文字
        int fontSize = 14; // 字体大小
        DrawText(GD.FontDefault, SelectedWindow.Title, titlePos, fontSize, textColor);

        // 可选：绘制窗口句柄/PID（在标题下方）
        Vector2 infoPos = new Vector2(SelectedWindow.Rect.X, SelectedWindow.Rect.Y - 5);
        string infoText = $"HWND: {SelectedWindow.Hwnd} | PID: {SelectedWindow.Pid}";
        DrawText(GD.FontDefault, infoText, infoPos, 12, Color.Yellow);
    }
}

/// <summary>
```

```
/// 选中窗口并触发重绘（外部调用）
/// </summary>
/// <param name="window">要选中的窗口</param>
public void SelectWindowAndRedraw(WindowInfo window)
{
    SelectedWindow = window;
    QueueRedraw(); // 触发_Draw函数，刷新绘制
}
#endregion
```

5.2 调用示例（选中Chrome第一个窗口并绘制）

在 `_Ready` 中添加：

Code block

```
// 查找Chrome窗口并选中绘制
var chromeWindows = FindWindowsByTitle("Chrome");
if (chromeWindows.Count > 0)
{
    SelectWindowAndRedraw(chromeWindows[0]);
    GD.Print($"已选中并绘制Chrome窗口：{chromeWindows[0].Title}");
}
```

运行后，Godot窗口中会显示Chrome窗口对应的红色半透明矩形，左上角有窗口标题和句柄/PID，和屏幕上Chrome的实际位置完全一致。

六、进阶功能4：实时更新窗口信息（适配窗口拖动/大小修改）

外部窗口的位置、大小、标题可能动态变化（如拖动Chrome窗口、修改记事本标题），需**定时重新枚举**窗口信息，保证数据实时性，通过 `_Process` 计时器实现，可自定义更新间隔。

6.1 新增实时更新变量和方法（添加到代码中）

6.1.1 新增全局变量（放在全局变量区域）

Code block

```
/// <summary>
/// 窗口信息更新计时器（秒）
/// </summary>
private float _updateTimer = 0f;
/// <summary>
/// 刷新间隔（秒），建议1-2秒，过短会占用性能
/// </summary>
private const float UpdateInterval = 1f;
```

6.1.2 新增 `_Process` 方法（实现定时刷新）

Code block

```
#region 进阶功能：实时更新窗口信息
public override void _Process(double delta)
{
    // 计时器累加
    _updateTimer += (float)delta;
    // 达到间隔，重新枚举窗口
    if (_updateTimer >= UpdateInterval)
    {
        _updateTimer = 0f;
        // 保存当前选中窗口的句柄，刷新后重新选中
        IntPtr selectedHwnd = SelectedWindow?.Hwnd ?? IntPtr.Zero;

        // 重新枚举外部窗口
        EnumerateExternalWindows();

        // 刷新后重新选中原窗口（保持绘制不变）
        if (selectedHwnd != IntPtr.Zero)
        {
            var newSelectedWindow = FindWindowByHwnd(selectedHwnd);
            if (newSelectedWindow != null)
            {
                SelectWindowAndRedraw(newSelectedWindow);
            }
            else
            {
                SelectedWindow = null;
                QueueRedraw(); // 窗口已关闭，清空绘制
            }
        }
    }
}
#endregion
```

6.2 效果

运行后，拖动/缩放外部选中的窗口，Godot中绘制的矩形会**每秒刷新一次**，和外部窗口的实际位置/大小保持一致；若外部窗口被关闭，Godot会自动清空绘制。

七、高阶拓展：按窗口类名精准查找（避免标题重复）

部分软件的窗口标题可能重复（如多个记事本窗口），但**窗口类名**是唯一的（如微信类名 `WeChatMainWndForPC`、Chrome类名 `Chrome_WidgetWin_1`），可通过 `FindWindow` API 按类名精准查找，无需全量枚举，效率更高。

7.1 封装类名查找API（添加到API封装区域）

Code block

```
// 按类名/标题精准查找单个窗口（无需枚举，效率最高）
[DllImport("user32.dll", CharSet = CharSet.Auto)]
private static extern IntPtr FindWindow(string lpClassName, string lpWindowName);
```

7.2 新增类名查找方法

Code block

```
#region 高阶拓展：按窗口类名查找（唯一标识）
/// <summary>
/// 按类名精准查找窗口（推荐，避免标题重复）
/// </summary>
/// <param name="className">窗口类名（通过Spy++工具获取）</param>
/// <returns>窗口句柄（无则返回IntPtr.Zero）</returns>
public IntPtr FindWindowByClassName(string className)
{
    return FindWindow(className, null); // 仅按类名查找，标题设为null
}

/// <summary>
/// 按类名+标题查找窗口（双重过滤，最精准）
/// </summary>
/// <param name="className">窗口类名</param>
/// <param name="windowName">窗口标题</param>
/// <returns>窗口句柄（无则返回IntPtr.Zero）</returns>
public IntPtr FindWindowByClassAndTitle(string className, string windowName)
{
    return FindWindow(className, windowName);
}
#endregion
```

7.3 类名获取工具：Spy++

- 类名需通过 **Spy++** 工具获取（Visual Studio 自带，可在VS中搜索「Spy++」）；

- 打开Spy++后，点击**查找窗口** → 拖动目标图标到外部窗口上 → 即可看到窗口的**类名（Class Name）**、句柄、PID等信息；
- 常用软件类名参考：
  - 微信：`WeChatMainWndForPC`
  - Chrome：`Chrome_WidgetWin_1`
  - 记事本：`Notepad`
  - 资源管理器：`CabinetWClass`

7.4 调用示例（按类名查找微信）

Code block

```
// 按类名查找微信主窗口，无需全量枚举
IntPtr weChatHwnd = FindWindowByClassName("WeChatMainWndForPC");
if (weChatHwnd != IntPtr.Zero)
{
    GD.Print($"通过类名找到微信窗口，句柄：{weChatHwnd}");
    ActivateWindow(weChatHwnd); // 直接激活
}
```

八、完整工程使用指南

1. **新建C# 2D工程**，创建节点结构（Node2D → WindowDetector + 可选Button）；
2. **创建WindowDetector.cs**，复制本文所有代码（按区域拼接，无冲突）；
3. **挂脚本+运行**，控制台查看枚举结果，Godot窗口中看到绘制的外部窗口矩形；
4. **按需调用方法**：
   - 全量枚举：`EnumerateExternalWindows()`；
   - 精准查找：`FindWindowsByTitle()` / `FindWindowByClassName()`；
   - 窗口交互：`ActivateWindow()`；
   - 可视化：`SelectWindowAndRedraw()`。

九、性能优化建议

1. **刷新间隔**：实时更新的 `UpdateInterval` 建议设为 **1-2秒**，过短（如0.1秒）会频繁调用系统API，占用CPU性能；
2. **避免重复枚举**：若无需实时更新，仅在需要时手动调用 `EnumerateExternalWindows()`，而非在 `_Process` 中循环；

3. **按需终止枚举**：若只需查找单个窗口，可在 `EnumWindowCallback` 中找到目标后返回 `false`，终止后续枚举，减少系统调用；
4. **过滤小窗口**：可在回调中添加窗口大小过滤（如排除宽高小于50的窗口），过滤掉系统托盘、小工具等无效窗口：

Code block

```
// 在回调函数中添加，封装WindowInfo前
if (godotRect.Size.X < 50 || godotRect.Size.Y < 50) return true;
```

十、常见问题排查

1. **Godot崩溃**：检查API封装的**委托签名、结构体布局、参数类型**是否和原生一致（如 `HWND` 必须是 `IntPtr`，`BOOL` 必须是 `bool`）；
2. **枚举不到窗口**：确保窗口是**顶层可视窗口**（子窗口如按钮/输入框无法枚举），且未被自身PID过滤；
3. **激活窗口失败**：部分软件（如游戏、管理员权限程序）会阻止外部激活，需将Godot以**管理员身份运行**；
4. **标题截断**：增大 `TitleBufferSize`（如改为512），避免长标题被截断；
5. **绘制位置偏移**：确保Godot窗口的**屏幕坐标**和系统一致，未开启多屏缩放（Windows显示设置→缩放→100%）。

以上方案覆盖了Windows平台Godot 4 C# 外部窗口识别的**所有核心场景**，代码可直接落地，拓展性强，可基于此实现**窗口捕获、窗口吸附、多软件联动、自动化操作**等高级功能。

（注：文档部分内容可能由 AI 生成）